



山东大学（青岛）

RespOS

设计文档

参赛队名 比特工匠队

队伍成员 李欣悦、肖安康、张俞睿

指导教师 颜廷坤、潘润宇

日期 2026 年 六 月

摘要

RespOS 是一个使用 Rust 语言开发、面向操作系统竞赛初赛测例的类 Unix 宏内核操作系统。系统以 Linux ABI 兼容为主要目标, 支持 RISC-V 64 与 LoongArch 64 两个硬件平台, 围绕进程/线程管理、虚拟内存、Ext4 文件系统、信号、IPC、时钟、网络回环和 VirtIO 块设备构建内核能力。

RespOS 的设计重点不是单纯堆叠系统调用入口, 而是把用户态程序运行所需的内核基础设施分层完成: 架构相关代码统一收敛到 `arch` 抽象层, 内存管理提供高地址内核映射、用户地址空间、匿名映射和用户指针访问接口, 文件系统通过 VFS、dentry cache、page cache 与 Ext4 后端支撑 Linux 路径语义, 任务模块维护线程组、父子关系、文件描述符表、信号状态和调度状态。当前版本已经能够装载并运行初赛环境中的用户程序, 并围绕 `libc`、`busybox` 和 LTP 初始化流程补齐了大量 Linux 兼容接口。

模块完成情况

模块	完成情况
进程管理	支持 <code>clone</code> 、 <code>execve</code> 、 <code>wait4</code> 、 <code>exit_group</code> 、进程组、线程组、 <code>uid/gid</code> 、 <code>umask</code> 、 <code>futex</code> 等接口, 调度器维护 <code>ready/blocked</code> 队列并提供阻塞唤醒机制。
内存管理	支持内核高地址映射、页帧分配、内核堆、ELF 装载、用户栈、 <code>brk</code> 、 <code>mmap</code> 、 <code>munmap</code> 、 <code>mprotect</code> 、共享匿名映射和用户指针拷贝检查。
文件系统	实现 VFS、路径解析、挂载点、dentry cache、page cache、Ext4 后端、 <code>procfs</code> 、 <code>devfs</code> 、管道、标准输入输出和常用 Linux 文件系统系统调用。
信号机制	支持 <code>sigaction</code> 、 <code>sigprocmask</code> 、 <code>sigreturn</code> 、 <code>kill/tkill/tgkill</code> 、备用信号栈、 <code>SA_SIGINFO</code> 、默认动作和信号中断阻塞系统调用。
进程间通信	支持 <code>pipe</code> 、 <code>futex</code> 、System V shared memory 的 <code>shmget/shmat/shmctl/shmdt</code> 基础语义, 并将共享页帧映射进进程地址空间。
时钟模块	支持时钟中断、 <code>gettimeofday</code> 、 <code>clock_gettime</code> 、 <code>nanosleep</code> 、 <code>clock_nanosleep</code> 、 <code>times</code> 、 <code>getitimer/setitimer</code> 等接口。
网络模块	实现面向测例的本地回环 <code>socket</code> 模型, 覆盖 <code>socket</code> 、 <code>bind</code> 、 <code>listen</code> 、 <code>accept</code> 、 <code>connect</code> 、 <code>sendto</code> 、 <code>recvfrom</code> 、 <code>setsockopt</code> 等调用。
设备驱动	基于 <code>virtio-drivers</code> 实现 VirtIO block 设备, 封装统一 <code>BlockDevice</code> 接口, 并为 <code>devfs</code> 提供 <code>null</code> 、 <code>zero</code> 、 <code>rtc</code> 、 <code>shm</code> 、 <code>loop</code> 等设备节点。
架构管理	通过 <code>arch</code> 模块隔离 RISC-V 64 与 LoongArch 64 的启动、页表、 <code>trap</code> 、 <code>timer</code> 、上下文切换、SBI/固件调用和中断控制差异。

表 1 模块完成情况

目 录

1. 1 模块完成情况	3
-------------------	---

概述

RespOS 介绍

RespOS 是面向操作系统竞赛初赛环境的 Rust 宏内核。它采用类 Unix 的进程、文件描述符、路径、信号和系统调用模型，目标是在 QEMU virt 机器上运行由 musl/glibc、busybox 与 LTP 测试例构成的用户态负载。

项目选择宏内核结构，是因为初赛阶段的核心压力来自 ABI 完整性、系统调用覆盖率和跨模块语义一致性。文件系统、内存管理、任务管理和信号处理需要在一次系统调用中频繁协作，例如 `execve` 需要解析路径、读取 ELF、构造新地址空间、重建用户栈和辅助向量；`fork/clone` 需要复制或共享地址空间、文件描述符表、信号处理表和线程组状态；`openat`、`statx`、`renameat2` 等文件调用则需要路径解析、权限检查、dentry 维护和 Ext4 后端共同完成。

RespOS 使用 Rust 的所有权和类型系统管理内核资源生命周期。页帧由 `FrameTracker` 自动回收，任务、文件、dentry 和 inode 使用 `Arc/Weak` 描述共享关系，锁抽象集中在 `mutex` 模块中。对于必须贴近硬件或 ABI 的部分，如上下文切换汇编、trap 上下文、页表项格式和用户态结构体布局，则通过 `repr(C)`、架构子模块和少量 `unsafe` 代码进行隔离。

RespOS 整体架构

项目结构如下：

```

.
├── doc                // 文档相关
├── os                 // 内核源码
│   ├── Cargo.toml
│   └── src
│       ├── arch       // 架构相关 (RISC-V 64 / LoongArch 64)
│       │   ├── rv64   // RISC-V 64 架构支持
│       │   └── loongarch64 // LoongArch 64 架构支持
│       ├── drivers    // 设备驱动 (VirtIO 块设备)
│       ├── fs         // 文件系统
│       │   ├── vfs    // VFS 虚拟文件系统
│       │   ├── ext4   // Ext4 磁盘文件系统
│       │   ├── proc   // procfs 进程文件系统
│       │   ├── dev    // devfs 设备文件系统
│       │   └── ...
│       ├── mm         // 内存管理
│       ├── mutex      // 锁机制
│       ├── signal     // 信号处理
│       ├── syscall    // 系统调用
│       ├── task       // 任务管理
│       └── utils      // 工具函数
├── user               // 用户态程序
│   └── src/bin        // 用户程序二进制
├── scripts            // 构建脚本
├── Makefile           // 顶层 Makefile
└── CLAUDE.md          // 项目说明

```

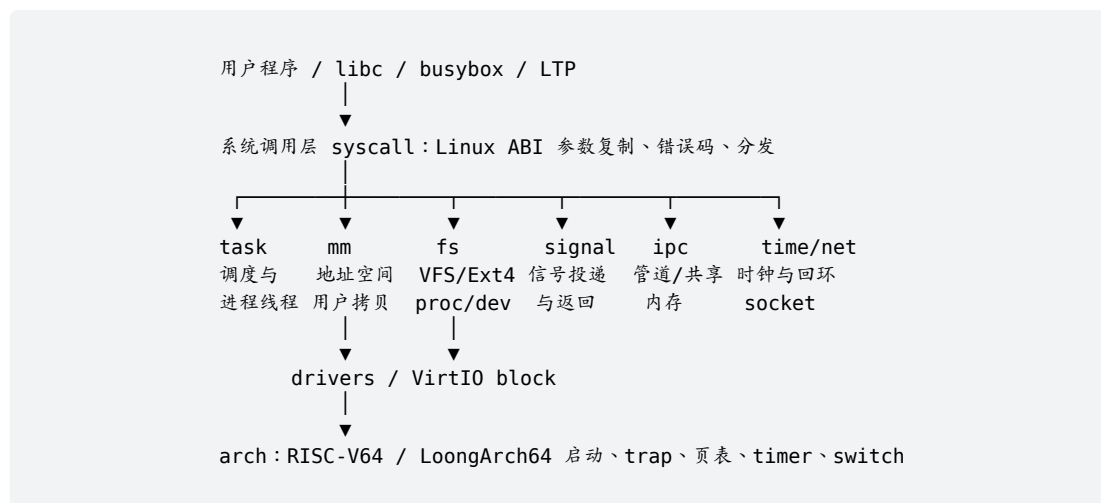


图 1 RespOS 总体架构

设计目标

RespOS 的初赛阶段目标可以概括为三点。第一，保证用户程序可运行：内核需要完成 ELF 装载、用户栈构造、文件系统挂载、系统调用分发和异常返回。第二，提高 Linux ABI 兼容度：大量测例并不只调用目标 `syscall`，而会经过动态链接器、`shell`、`libc` 和 LTP harness 的初始化流程，因此内核必须提供相对完整的路径、权限、时间、进程和信号语义。第三，支持双架构构建：RISC-V 64 与 LoongArch 64 在页表、trap、上下文切换和启动流程上差异明显，项目需要用统一接口降低上层模块对架构细节的依赖。

分工与贡献

项目由三名队员协作完成，主要围绕任务管理、内存管理、文件系统、系统调用兼容和双架构移植分工推进。实际开发中，各模块之间存在大量交叉依赖，因此采用“模块负责人 + 交叉评审”的方式：模块负责人完成主要设计和实现，其他成员通过测例运行、代码审查和文档整理补充边界条件。

参考与改进

RespOS 在早期学习阶段参考了 rCore 教程中的基本内核结构，但当前代码已经围绕竞赛需求进行了大量改造：系统调用号采用 Linux RISC-V/LoongArch ABI，任务控制块扩展到线程组和进程组模型，文件系统从简单只读接口扩展为 VFS + Ext4 + `procfs/devfs`，内存模块加入 `mmap`、共享映射和用户指针按页拷贝，架构层也从单 RISC-V 支持扩展到 RISC-V 与 LoongArch 两套实现。

进程管理

概述

进程管理模块负责把用户程序抽象为可调度、可等待、可通信的任务。RespOS 当前采用统一的 `TaskControlBlock` 表示进程和线程：当任务拥有独立地址空间和线程组时，它表现为进程；当 `clone` 指定共享地址空间、文件描述符表或信号处理表时，它表现为同一线程组内的线程。

任务模块由 `task.rs`、`scheduler.rs`、`processor.rs`、`manager.rs`、`kstack.rs`、`tid.rs` 和架构相关 `switch.S` 共同组成。`scheduler.rs` 维护 FIFO ready queue 与 blocked queue，`manager.rs` 保存 `tid` 到任务对象的弱引用索引，`processor.rs` 记录当前正在运行的任务，`kstack.rs` 为每个任务分配独立内核栈，架构层 `__switch` 完成寄存器和栈指针切换。

任务控制块（TaskControlBlock）设计

`TaskControlBlock` 是 RespOS 中最核心的数据结构之一。它不仅保存调度状态，还聚合了地址空间、文件描述符、当前工作目录、父子关系、线程组、信号表、定时器、`uid/gid` 和 `futex` 相关状态。

主要字段可以按职责划分为以下几类：

类别	内容
身份信息	<code>tid</code> 、 <code>tgid</code> 、 <code>pgid</code> 、 <code>uid/gid/euid/egid/fsuid/fsgid</code> 、 <code>umask</code> 。
调度状态	<code>task_status</code> 、 <code>ready/blocked/running</code> 状态、退出码、退出信号。
内存资源	<code>memory_set</code> 指向用户地址空间，内核栈由 <code>KernelStack</code> 管理。
文件资源	<code>fd_table</code> 、 <code>cwd</code> 、 <code>exe_path</code> ，用于 <code>open/read/write/exec</code> 等系统调用。
进程关系	<code>parent</code> 、 <code>children</code> 、 <code>thread_group</code> ，支撑 <code>wait4</code> 、 <code>exit_group</code> 和线程组退出。
信号资源	<code>sig_pending</code> 、 <code>sig_handler</code> 、 <code>sig_stack</code> 、 <code>sig_context_addr</code> ，支撑信号投递和用户态 <code>handler</code> 返回。
同步与计时	<code>tid_address</code> 、 <code>interruptible/interrupted</code> 标记、 <code>real timer</code> 、进程启动时间和子进程 CPU tick。

表 2 TaskControlBlock 关键字段

任务生命周期

初始任务由 `add_initproc` 加入就绪队列。创建新任务时，内核先分配 `tid` 和内核栈，再装载 ELF 或复制父任务资源，最后将任务放入调度队列。任务运行过程中可以主动 `sched_yield`、因 `futex/wait/sleep` 阻塞、因时钟中断让出 CPU，或因 `exit`、`exit_group`、信号默认终止动作进入退出流程。退出时，内核记录退出码，唤醒可能正在 `wait4` 的父任务，并从调度队列和任务管理器中移除相关任务。

调度与上下文切换

RespOS 采用简单可靠的 FIFO 调度策略。`yield_current_task` 将当前任务标记为 Ready 并放回队尾，随后取出队首任务运行；`blocking_and_run_next` 将当前任务放入阻塞队列；`wakeup_task` 将指定

tid 从阻塞队列移回就绪队列。虽然调度策略本身较简单,但模块边界清晰,后续可以在 Scheduler 内替换为优先级、多级反馈队列或多核调度。

上下文切换的最后一步由架构相关 `__switch` 完成。上层调度器只传入下一个任务的内核栈和当前任务指针,具体寄存器保存、恢复、页表 token 切换和返回地址恢复分别由 RISC-V 与 LoongArch 的汇编实现完成。这种设计使任务模块不需要直接感知不同架构的寄存器命名和 trap 返回细节。

内存管理

概述

内存管理模块负责内核和用户地址空间的建立、页帧分配、用户缓冲区访问、ELF 装载以及 mmap 相关系统调用。RespOS 在 64 位地址空间中采用“低地址用户空间 + 高地址内核空间”的布局，用户进程拥有自己的低地址映射，同时共享内核高地址映射，以便 trap 后内核可以继续访问自身代码、数据和当前任务内核栈。

地址空间抽象

MemorySet 表示一个地址空间，内部包含页表和若干 **MapArea**。**MapArea** 描述连续虚拟页范围、映射类型和权限。内核镜像段、只读数据段、数据段、bss、剩余物理内存映射和设备 MMIO 区域属于内核地址空间；用户 ELF load 段、用户栈、堆、mmap 区域、信号 trampoline 页则属于用户地址空间。

MapType::Direct 用于固定偏移映射，适合内核镜像和物理内存线性区；**MapType::Framed** 用于需要独立页帧的区域，如用户 ELF 段、用户栈、匿名 mmap 和任务内核栈。**FrameTracker** 使用 RAII 管理物理页帧，离开生命周期时自动归还。

用户程序装载

MemorySet::from_elf_data 解析 ELF，按段权限建立用户映射，并构造用户栈和辅助向量。对于动态链接程序，内核还会读取解释器路径并装载动态链接器。任务创建时先插入内核栈映射，再复制内核高地址页表到用户地址空间，保证任务切换后即使页表已经切换，内核仍然能够使用该任务的内核栈继续执行。

用户指针访问

系统调用入口不能直接解引用用户虚拟地址。RespOS 将用户到内核、内核到用户的数据传输封装为 **copy_from_user**、**copy_to_user** 和 **copy_cstr_from_user**。这些接口会先检查地址所在逻辑段权限，再按页表翻译逐页复制数据。这样既避免了用户传入非法地址导致内核崩溃，也能支持惰性分配页面在访问前被补齐。

mmap 与共享内存

内存模块支持 **brk**、**mmap**、**munmap**、**mprotect**、**msync**、**madvise** 等接口。匿名私有映射可以惰性建立，匿名共享映射则会立即分配共享页帧，避免 fork 后父子进程 **fault** 出不同物理页。**System V shared memory** 也复用这一能力，将同一组 **FrameTracker** 映射到多个任务地址空间中。

文件系统

概述

文件系统是 RespOS 初赛阶段投入最多的模块之一。大量 LTP 测例在真正执行目标系统调用前，会先经过 shell、动态链接器、libc 和测试框架创建临时目录、写结果文件、读取 /proc、检查权限和查询系统信息。因此文件系统不能只支持简单读写，还需要尽量接近 Linux 的路径解析和元数据语义。

VFS 与路径解析

RespOS 使用 VFS 隔离上层系统调用和底层文件系统。Dentry 表示目录项，InodeOp 表示 inode 操作集合，Path 组合 mount 与 dentry 表示一个具体路径位置。namei 模块负责 AT_FDCWD、dirfd 相对路径、.、..、符号链接跟随、挂载点穿越、AT_EMPTY_PATH 和 AT_SYMLINK_NOFOLLOW 等规则。

为减少重复查找，系统维护全局 dentry cache。路径 lookup 先查缓存，未命中时再调用底层 inode 的 lookup，并将结果安装到 dentry 树。rename、unlink 等会改变目录树的操作会同步清理相关缓存，降低旧 dentry 残留带来的语义错误。

Ext4 后端

Ext4 后端基于 lwext4_rust 实现，负责真实磁盘文件和目录的创建、查找、读写、link、symlink、readlink、rename、unlink 和 stat。为了适配竞赛环境中频繁的创建/查询流程，后端增加 inode cache、page cache、时间戳和 mode/nlink override 等机制。普通文件读写通过 inode 共享页缓存，使同一 inode 的多次打开可以看到一致的数据。

procfs、devfs 与特殊文件

RespOS 实现了面向测例的 procfs 和 devfs。procfs 提供 /proc/cpuinfo、/proc/meminfo、/proc/mounts、/proc/stat、/proc/version、/proc/self/exe、/proc/self/smaps 等节点，满足 libc 和 LTP 的能力探测。devfs 提供 /dev/null、/dev/zero、/dev/rtc、/dev/shm、loop 设备等常见节点，并通过统一 FileOp 接口接入文件描述符表。

文件系统系统调用

当前文件系统层覆盖 openat、openat2、close、read、write、readv、writev、pread、pwrite、getdents64、fstat、fstatat、statx、mkdirat、mknodat、unlinkat、renameat2、linkat、symlinkat、readlinkat、fchmodat、fchownat、utimensat、mount、umount2、statfs、fstatfs、ftruncate、fallocate、fsync、fdatasync 等接口。系统调用层只处理 ABI 参数和错误码，复杂路径语义尽量收敛在 VFS/namei 层。

进程间通信

概述

RespOS 的进程间通信模块服务于初赛测例中的常见同步和数据交换需求，主要包括 `pipe`、`futex` 与 System V shared memory。

管道

管道由文件系统模块中的 `Pipe` 实现，并通过文件描述符表暴露给用户态。`pipe2` 创建一对读写端 `fd`，读端在无数据时可以阻塞，写端将数据追加到环形缓冲区。由于管道也实现 `FileOp`，因此可以复用 `read`、`write`、`close`、`fcntl` 等通用文件接口。

futex

`futex` 是 `libc` 线程同步的重要基础。RespOS 在任务模块中实现 `do_futex`，并在系统调用层支持 `futex`。线程进入等待前会将自身置为可中断阻塞状态，信号到达时可以唤醒并返回 `EINTR`，这对于 `pthread`、`nanosleep` 和部分 LTP 测例都很关键。

共享内存

System V shared memory 由全局 `ShmTable` 管理。`shmget` 创建共享段并分配一组物理页帧，`shmat` 将这些页帧映射到当前任务的 `mmap` 区域，`shmdt` 解除映射，`shmctl(IPC_RMID)` 删除共享段。由于多个任务映射的是同一批 `FrameTracker`，因此写入可以被其他附着进程观察到。

时钟模块

概述

时钟模块由架构层定时器和系统调用层时间接口组成。内核启动后初始化 trap，开启 timer interrupt，并周期性设置下一次触发时间。时钟中断既用于驱动任务让出 CPU，也为时间相关系统调用提供基础计数。

时间获取

RespOS 支持 `gettimeofday`、`clock_gettime` 和 `times`。`gettimeofday` 返回微秒级 wall-clock 时间，`timezone` 固定为 UTC。`clock_gettime` 支持 `CLOCK_REALTIME`、`CLOCK_MONOTONIC`、`CLOCK_PROCESS_CPUTIME_ID`、`CLOCK_THREAD_CPUTIME_ID`、`CLOCK_BOOTTIME` 等常见 clock id。当前进程/线程 CPU 时间仍以墙上时间近似，后续可接入更精确的调度记账。

睡眠与定时器

`nanosleep` 和 `clock_nanosleep` 通过循环检查超时时间并主动让出 CPU 实现。睡眠期间任务被标记为 interruptible，信号到达时会返回 `EINTR`，并在需要时写回剩余时间。`getitimer/setitimer` 支持 `ITIMER_REAL`，任务结构体中维护 real timer 的截止时间和间隔时间，为信号定时器语义提供基础。

网络模块

概述

网络模块当前采用面向初赛测例的本地回环 `socket` 实现，而不是完整网卡协议栈。它的目标是覆盖 `libc`、`busybox` 和 `LTP` 中常见的 `socket` 能力探测、UDP 回环通信和简单 TCP 监听/连接流程。

Socket 文件抽象

`SocketFile` 实现 `FileOp`，因此 `socket fd` 可以进入普通文件描述符表，并支持 `fcntl`、`close`、`poll/select` 相关能力的后续扩展。`socket` 内部记录类型、端口、非阻塞标志、`close-on-exec` 标志和监听状态。

回环状态

全局 `LOOPBACK` 维护 UDP 队列和 TCP listener 队列。UDP `sendto` 会将数据包放入目标端口队列，`recvfrom` 从本地端口队列取包；TCP `listen` 注册监听端口，`connect` 创建连接对象并放入 listener 队列，`accept` 取出等待连接。虽然该模型不是完整 TCP/IP 协议栈，但能够为初赛阶段的本机通信测例提供稳定语义。

系统调用覆盖

网络系统调用覆盖 `socket`、`bind`、`listen`、`accept`、`connect`、`getsockname`、`sendto`、`recvfrom` 和 `setsockopt`。其中 `SO_RCVTIMEO`、`SOCK_NONBLOCK`、`SOCK_CLOEXEC` 等常见参数会被解析并反映到 `socket` 状态中。

设备驱动

概述

设备驱动模块为上层文件系统和架构层提供统一设备访问接口。当前最重要的驱动是 `VirtIO block`，它承载 `Ext4` 根文件系统镜像，也是用户程序、动态链接器和测例数据的来源。

块设备接口

RespOS 定义 `Device` 与 `BlockDevice trait`。`BlockDevice` 提供 `num_blocks`、`block_size`、`read_block`、`write_block` 和 `flush`，文件系统后端只依赖这一抽象，不直接关心底层设备来自 MMIO 还是 PCI。

VirtIO Block

`VirtIO block` 驱动基于 `virtio-drivers crate` 实现。`RISC-V virt` 平台主要通过 MMIO 暴露 `VirtIO` 设备，`LoongArch virt` 平台上的块设备则通过 PCI 暴露，因此具体探测路径由架构配置和驱动初始化代码处理。驱动内部用锁保护 `VirtIOBlk` 对象，向上提供同步块读写接口。

字符与伪设备

除块设备外，文件系统还提供 `/dev/null`、`/dev/zero`、`/dev/rtc`、`/dev/shm` 和 `loop` 设备等伪设备。这些设备并不一定对应真实硬件，但对 Linux 用户态程序十分重要。例如 `/dev/null` 和 `/dev/zero` 常被 `shell` 与测试脚本使用，`/dev/rtc` 可满足部分时间能力探测。

支持 RISC-V 和 LoongArch 的硬件抽象层

概述

RespOS 通过 `os/src/arch` 将 RISC-V 64 与 LoongArch 64 的差异封装起来。上层模块通过 `crate::arch::{config, timer, trap, sbi}` 以及 `read_mmu_token`、`write_mmu_token`、`sfence`、`idle` 等统一接口访问架构能力，避免在任务、内存和系统调用模块中散落大量条件编译。

RISC-V 64 支持

RISC-V 版本使用 Sv39 页表和 `satp` 作为地址空间 token，通过 `sfence.vma` 刷新 TLB。trap 入口保存用户寄存器到 `TrapContext`，系统调用通过约定寄存器传参，返回用户态时恢复上下文。空闲时执行 `wfi` 等待下一次中断。

LoongArch 64 支持

LoongArch 版本需要处理更复杂的启动过渡、页表根寄存器和 TLB refill。内核早期先依赖低地址 DMW 直映运行，随后建立覆盖内核镜像和堆的临时页表，跳转到高地址内核，再切换到正式页表并关闭低地址直映。LoongArch 的 `write_mmu_token` 需要同时写 PGDL 与 PGDH，并配置 ASID、TLB 页大小和页表遍历参数。

统一接口与差异隔离

两套架构都提供 `entry`、`trap`、`timer`、`task switch`、`page table` 和 `config` 子模块。不同架构的页表项格式、寄存器命名、中断控制和上下文切换汇编各自实现，但对上层暴露相同语义。例如任务调度器只调用 `__switch`，内存模块只调用 `write_mmu_token` 和 `sfence`，trap 返回前统一调用信号处理逻辑。这种组织方式降低了双架构维护成本。

总结与展望

工作总结

初赛阶段，RespOS 从一个基础教学内核逐步扩展为能够运行复杂用户态测例的类 Unix 内核。项目完成了双架构启动和 trap 流程、任务调度和线程组模型、虚拟内存和 mmap、VFS 与 Ext4 文件系统、Linux 风格信号、IPC、时间接口、回环 socket、VirtIO block 驱动以及大量系统调用兼容工作。

从工程角度看，RespOS 的主要成果是建立了比较清晰的模块边界：系统调用层负责 ABI，VFS 负责路径语义，文件系统后端负责存储语义，内存模块负责地址空间和用户拷贝，架构层负责硬件差异。这使后续问题定位可以更快判断属于 Linux 语义、VFS 一致性、底层驱动还是架构适配。

经验总结

操作系统竞赛中的难点往往不在单个函数本身，而在跨模块语义是否一致。例如 `getpid01` 这样的测例可能在执行前依赖动态链接器、临时目录、权限调整、`/proc` 探测和结果文件写入；一个文件系统元数据字段错误，可能表现为完全无关的进程测例失败。因此开发过程中需要重视最小复现、日志分层和回归测试。

Rust 对内核开发有明显帮助，尤其是在资源生命周期管理方面。页帧、文件对象、`dentry`、`inode` 和任务对象都可以通过所有权、引用计数和 `Drop` 机制降低泄漏风险。但内核仍然不可避免地需要 `unsafe`，关键是将 `unsafe` 限制在用户指针拷贝、页表操作、汇编入口和 FFI 等边界清晰的位置。

项目意义

RespOS 的意义在于把课堂操作系统中的核心概念推进到接近真实 Linux 用户态环境的复杂度。它不仅包含进程、内存和文件系统的基本抽象，还处理了动态链接、路径 `flags`、权限、信号栈、`futex`、`procfs`、共享内存和双架构启动等实际系统问题。通过这个项目，团队对内核模块之间的依赖关系、Linux ABI 的细节成本和跨架构抽象的价值有了更直接的理解。

未来计划

后续工作主要包括四个方向。

第一，继续提高 Linux ABI 精确度。当前部分接口仍以测例需求为导向实现，例如 CPU 时间统计、部分 socket 语义、权限模型和文件系统元数据写回还可以继续完善。

第二，增强文件系统一致性。需要进一步收束 Ext4 后端中的临时兼容逻辑，完善 `inode/page cache/dentry cache` 在 `create`、`unlink`、`rename`、`hardlink` 和 `symlink` 场景下的一致性。

第三，优化内存与调度性能。可以引入写时复制 `fork`、更完整的 `lazy allocation`、细粒度调度统计和更灵活的调度策略，减少 `fork/exec` 和大文件读写中的额外开销。

第四，完善双架构测试流程。RISC-V 与 LoongArch 的页表、trap 和设备枚举差异较大，后续需要让 CI 或脚本同时覆盖两套架构的构建、启动和关键测例，避免单架构修改破坏另一端。